SCOPE0

UI Testing Guide

Step-by-step walkthrough for testing every major workflow

Agent Registration · Tools · Policies · Approvals · API Integration · Guardrails

Your Login Credentials

Enter your Scope0 credentials below for reference during testing.

Email address

Password

Organization slug

API Key (created in Step 2 — store securely)

Prerequisites

- Scope0 is deployed and accessible (Railway or local Docker)
- You are signed in as an **owner** or **admin**
- You have your credentials ready (fill in the box on the cover page)

1 Register an Agent

- Click **Agents** in the left sidebar
- Click **+ New Agent** (top right)
- Fill in: Name, Slug (auto-filled), Description, Mode, Model, Daily action budget
- Click **Create Agent**
- On the agent detail page — copy the **Agent ID** (UUID shown next to the copy icon, or from the browser URL bar)

> *Save the Agent ID — you will need it when calling the Decision API in Step 8.*

2 Create an API Key

- Click **Settings** → **API Keys** tab
- Click **+ New API Key**, give it a name (e.g. *Test Key*)
- Set role to **developer** or **admin** → click **Create**
- **Copy the key immediately** — it is shown only once
- Write it in the credentials box on the cover page for reference

3 Register Tools

- Click **Tools** in the left sidebar → click **+ New Tool**
- Fill in: Namespace, Name, Description, Risk class, Input schema (JSON), Scope
- Click **Save**

Example — high-risk tool:

Namespace	`payments`
Name	`refund.issue`
Risk class	`critical`
Scope	`payments.refund.issue`

Example input schema:

```
{
```

```
  "type": "object",
  "properties": {
    "payment_id": { "type": "string" },
    "amount_cents": { "type": "integer" }
  }
}
```

4 Create a Policy

- Click **Policies** in the left sidebar → click **+ New Policy**
- Fill in: Name, Effect (allow / deny / require_approval), Priority, Actions, Condition (JSON)
- Click **Save**

Example — deny off-hours refunds:

Effect	`require_approval`
Priority	`100`
Actions	`payments.refund.issue`

```
{
  "op": "not",
  "args": [
    { "op": "time_between", "args": ["ctx.context.time", "09:00", "18:00"] }
  ]
}
```

5 Bind the Policy to an Agent

- Open the policy → click **Bindings** tab → click **+ Add Binding**
- Set Subject selector to `*` (all agents) or `agent:` (one agent)
- Click **Save Binding**

6 Assign the Agent a Role

- Click **Agents** → open your test agent

- Click **Assignments** tab → click **+ Add Assignment**
- Select a **User** and a **Role**
- Click **Save** — the agent now inherits the scopes attached to that role

7 Simulate a Policy Decision

- Click **Decisions** in the left sidebar
- Click **Simulate** or **+ New Decision**
- Select your test agent, enter action, resource type, and context JSON
- Click **Evaluate** — result shows allow / deny / require_approval with the matching policy

Example context to trigger the off-hours rule:

```
{ "ip": "1.2.3.4", "time": "03:00" }
```

8 Call the Decision API from Your Agent

This is how a real agent integrates with Scope0 over HTTP:

Endpoint and headers:

```
POST /api/v1/decisions

Authorization: Bearer

Content-Type: application/json
```

Request body:

```
{

  "subject_type": "agent",

  "subject_id": "",

  "action": "payments.refund.issue",

  "resource": {

    "type": "payment",

    "id": "pay_123",

    "attrs": {}

  },

  "context": {

    "ip": "1.2.3.4",

    "time": "03:00",

    "user_id": "usr_abc"

  },

}
```

Expected responses:

Scenario	Response
Action allowed	`{ "decision": "allow" }`
Denied by policy	`{ "decision": "deny", "reason": "..." }`

Approval required	`{ "decision": "require_approval", "approval_id": "..." }`

9 Approve or Deny a Pending Request

- Click **Approvals** in the left sidebar
- Find the pending request — shows agent name, action, resource, context, and inputs
- Review which policy triggered the approval requirement
- Click **Approve** or **Deny**
- The agent polls `GET /api/v1/approvals/` to check status and proceed

10 View the Audit Log

- Click **Audit** in the left sidebar
- Every decision, approval, and policy evaluation is logged with timestamp, agent, action, outcome, and matching policy
- Use filters to narrow by agent, date range, or outcome

11 Set Up a Guardrail

- Click **Guardrails** in the left sidebar
- Click **+ New Integration** to register your agent's event source
- Copy the **Webhook URL** shown — your agent POSTs events to this URL
- Click **Rules** tab → **+ New Rule**
- Set threshold (e.g. 5 events), time window (e.g. 5 minutes), severity filter, and action (revoke)
- Click **Save Rule**

> *When your agent fires 5+ high/critical events within 5 minutes, Scope0 automatically revokes its access.*

12 Manage Members

- Click **Settings** → **Members** tab
- To **invite**: click **+ Invite member**, enter email, select role, click **Send**
- To **change role**: click the role badge next to any member → pick new role from the dropdown
- To **remove**: click **Remove** in the Actions column → confirm the dialog

13 SSO Configuration (Enterprise)

- Click **Settings** → **SSO** tab → click **+ New SSO Connection**
- Choose provider: Okta, Google Workspace, Azure AD, or Generic SAML
- Follow the provider-specific instructions shown in the UI

- Test the connection before enabling it

Troubleshooting

Symptom	Fix
Sign-in fails	Check backend is running; verify DATABASE_URL in Railway points to the correct Postgres instance
Tools page blank	Seed script did not run; check backend logs for "Seeded demo org"
Decision always allows	No policy bindings — go to Policies and add a binding to `*`
Approval never appears	Policy effect must be `require_approval`, not `allow` or `deny`
API returns 401	API key is wrong or revoked; generate a new one in Settings → API Keys
Database empty	Hit `/api/v1/debug/db` — shows which Postgres the backend is connected to and row counts per table
Frontend 404 on /api/	BACKEND_URL env var not set in Railway frontend service